

제네릭클래스

: 데이터 타입을 타입매개변수로 사용하여 객체를 생성할 때 타입을 지정하도록 하는 기능 !!

=>타입의 안정성 확보 (컴파일 시점에 타입이 결정됨)

=>형변환없이 타입사용가능

```
public class Box {  
  
    private Object item;  
  
    public void setItem(Object item) {  
        this.item = item;  
    }  
  
    public Object getItem() {  
        return item;  
    }  
}
```

```
public class Main {  
  
    public static void main( String[] args){  
        Box box =new Box();  
        box.setItem( new Acorn());  
        Acorn acorn = (Acorn)box.getItem();  
  
        Box box2 = new Box();  
        box2.setItem( "문자열담올래");  
        String str = (String) box2.getItem();  
    }  
}
```

숫자 전용박스

```
public class BoxInteger {  
  
    private Integer item;  
  
    public void setItem(Integer item) {  
        this.item = item;  
    }  
  
    public Integer getItem() {  
        return item;  
    }  
}
```

```
public class Main {  
  
    public static void main( String[] args){  
        BoxInteger box =new BoxInteger();  
        box.setItem( 5);  
        Integer su= box.getItem();  
    }  
}
```

문자열 전용박스

```
public class BoxString {  
  
    private String item;  
  
    public void setItem(String item) {  
        this.item = item;  
    }  
  
    public String getItem() {  
        return item;  
    }  
}
```

```
public class Main {  
  
    public static void main( String[] args){  
        BoxString box =new BoxString();  
        box.setItem( "문자열만 담을래");  
        String str= box.getItem();  
    }  
}
```

```
public class Box<T> {  
  
    private T item;  
  
    public void setItem(T item) {  
        this.item = item;  
    }  
  
    public T getItem() {  
        return item;  
    }  
}
```

```
public class Main {  
    public static void main( String[] args){  
        Box<String> box =new Box<>();  
        box.setItem( "문자열만 담을래");  
        String su= box.getItem();  
  
        Box<Acorn> box2 = new Box<>();  
        box2.setItem( new Acorn("권지연"));  
        Acorn acorn = box2.getItem();  
    }  
}
```

제네릭 매서드 : 매서드 안에서 타입을 유연하게 지정해서 재사용성을 높일 수 있다

```
public class PrinterEx {  
  
    public static void printString(String data) {  
        System.out.println(data);  
    }  
  
    public static void printInteger(Integer data) {  
        System.out.println(data);  
    }  
  
    public static void printAcorn( Acorn acorn){  
        System.out.println( acorn);  
    }  
}
```

```
public class Main{  
  
    public static void main( String[] args){  
  
        PrinterEx.printString("문자열 파이팅");  
        PrinterEx.printInteger(7);  
        PrinterEx.printAcorn( new Acorn("김민환"));  
  
    }  
}
```

```
public class PrinterEx2 {  
  
    public static<T> void print( T data) {  
        System.out.println(data);  
    }  
  
}
```

```
public class Main{  
  
    public static void main( String[] args){  
  
        PrinterEx2.<String>print("문자열 파이팅");  
        PrinterEx2.<Integer>print(7);  
        PrinterEx2.<Acorn>print( new Acorn("김민환"));  
  
    }  
}
```